

# content

## PRAT 1

### Overview

General Intro of Our  
Search Engine

## PRAT 2

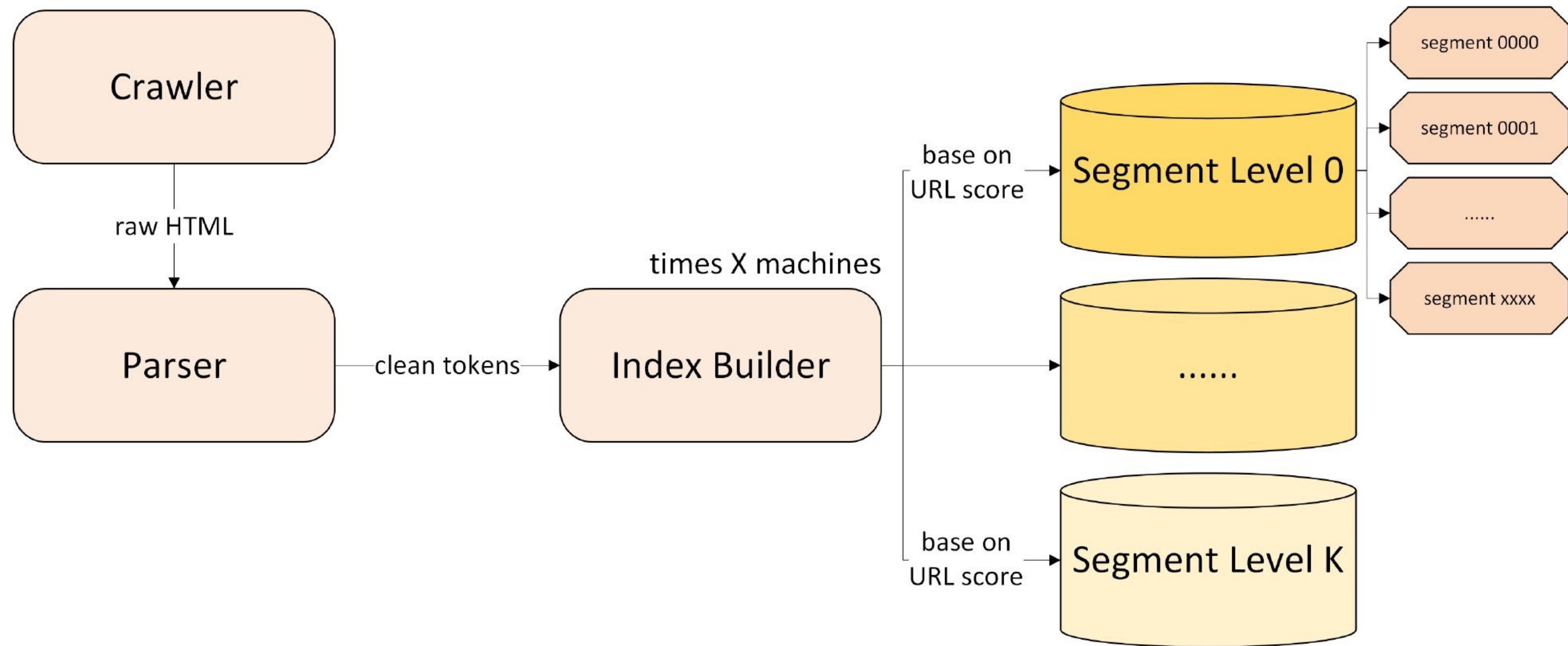
### Introduce by Part

Detailed Introduction  
Part by Part

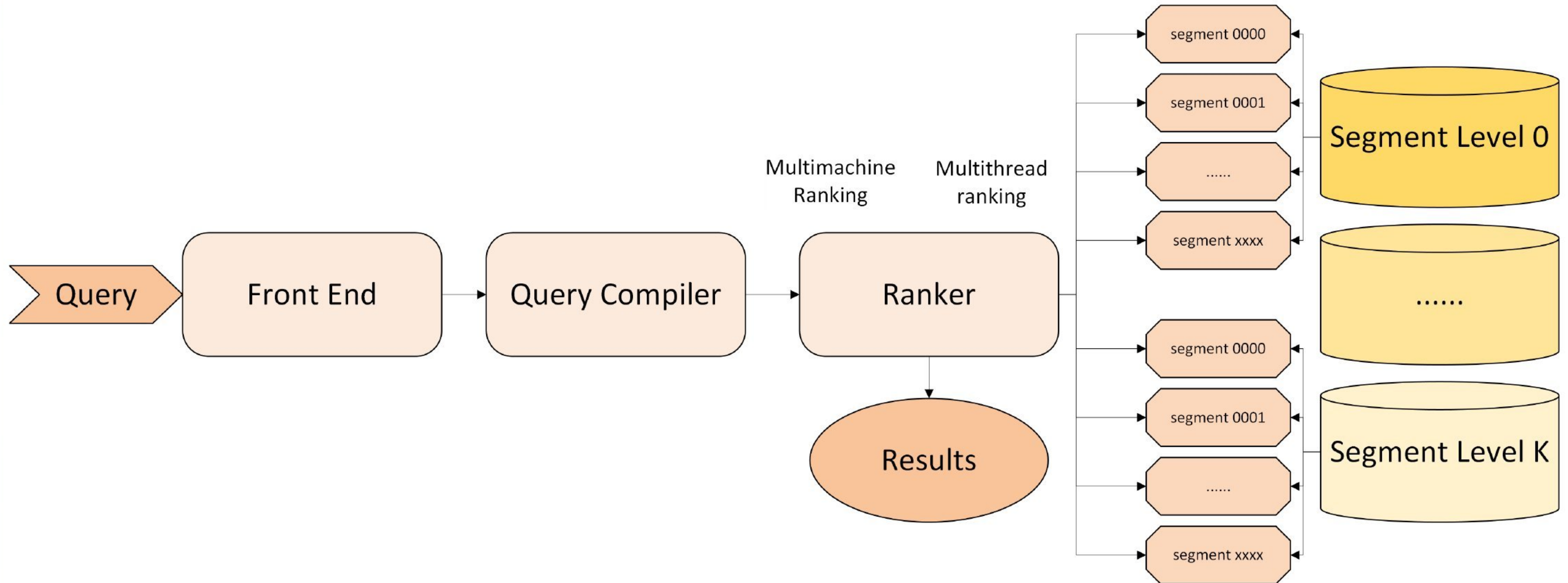
## PRAT 3

### Demo

Demo of the Result of  
Our Search Engine



# Offline Index Construction



# Online Query Serving

# Line of Code

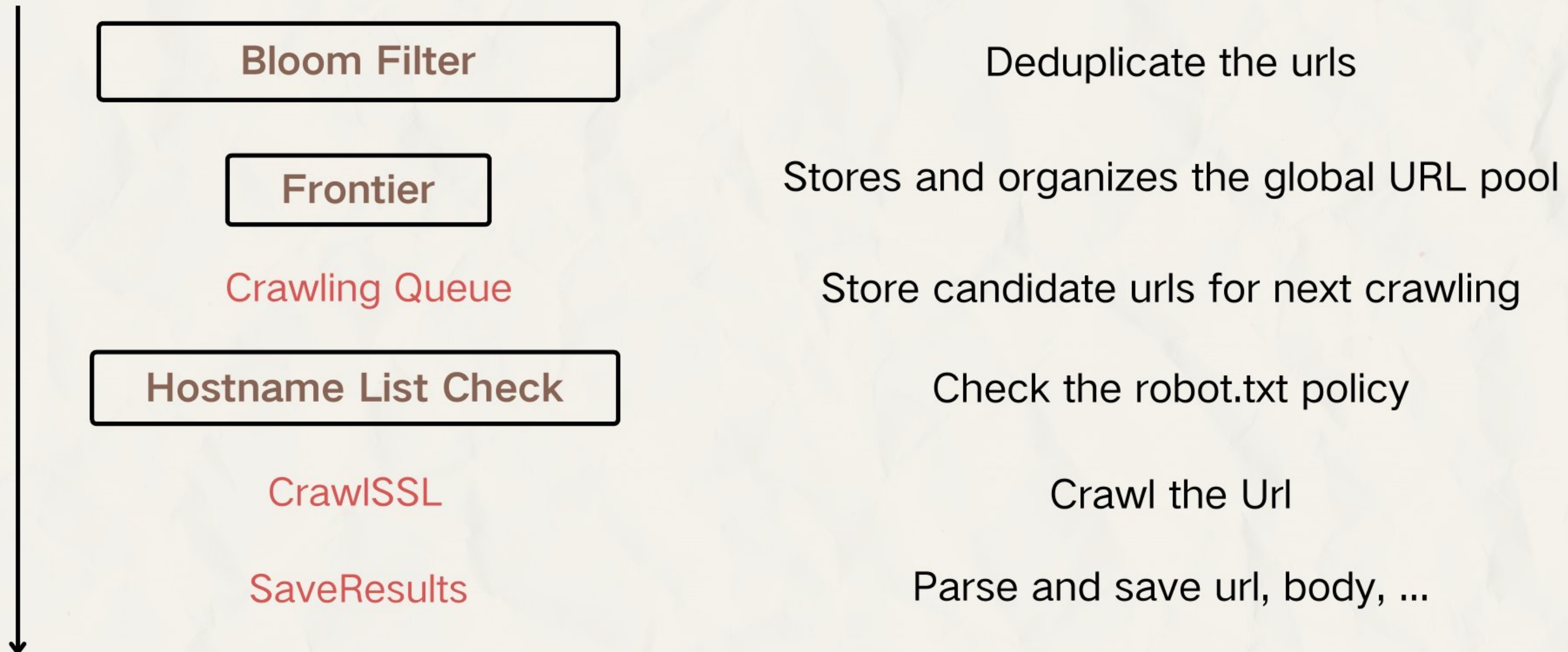
| Parts            | Line of Code |
|------------------|--------------|
| Common functions | ~1900        |
| Crawler          | ~4500        |
| Parser           | ~1200        |
| Index Builder    | ~5100        |
| Query Compiler   | ~770         |
| Ranker           | ~3100        |
| Frontend         | ~500         |
| Backend          | ~2400        |
| Total            | ~21000       |

# **Section 1**

## **Offline Index Construction**

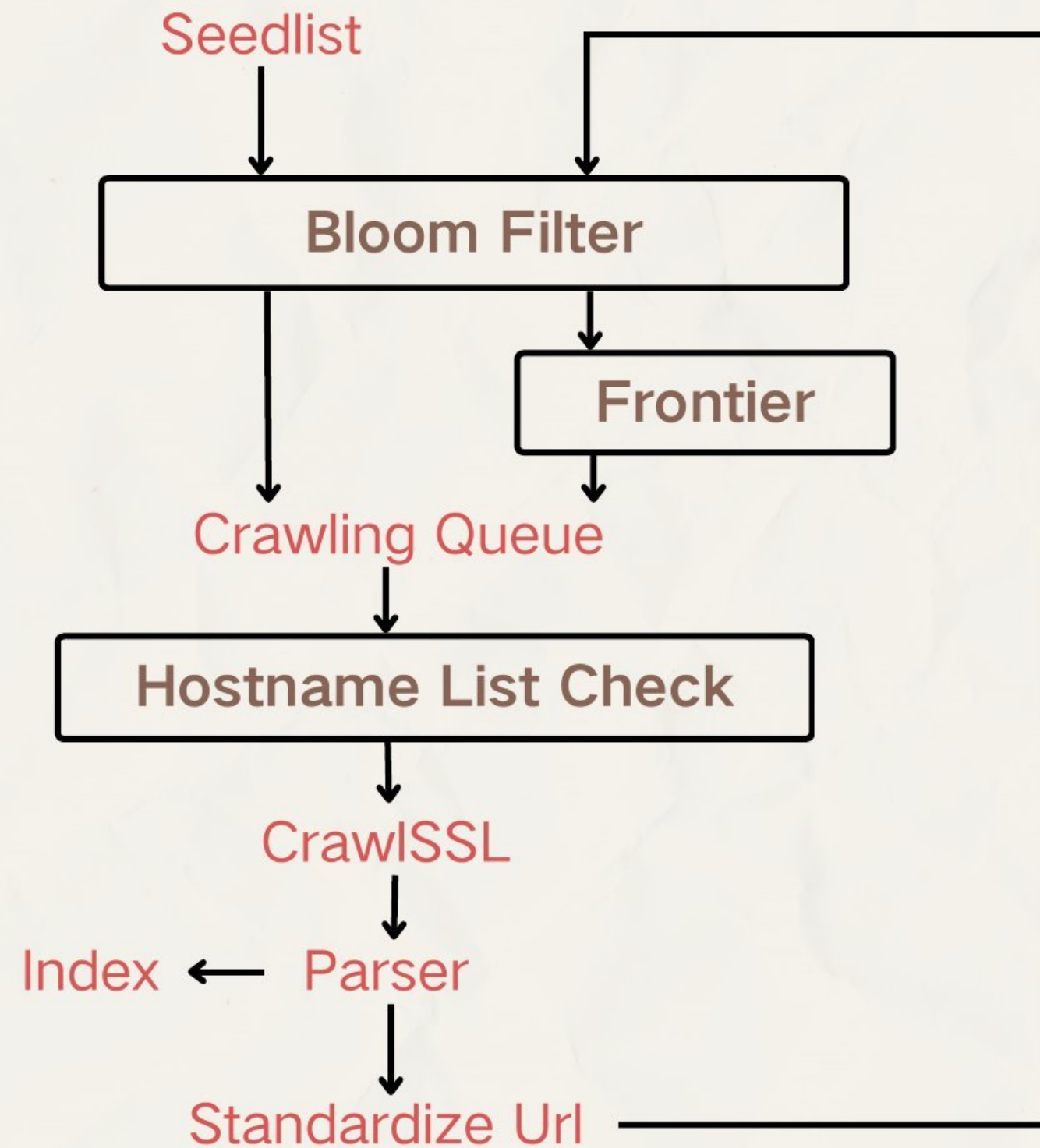
# Components

each url

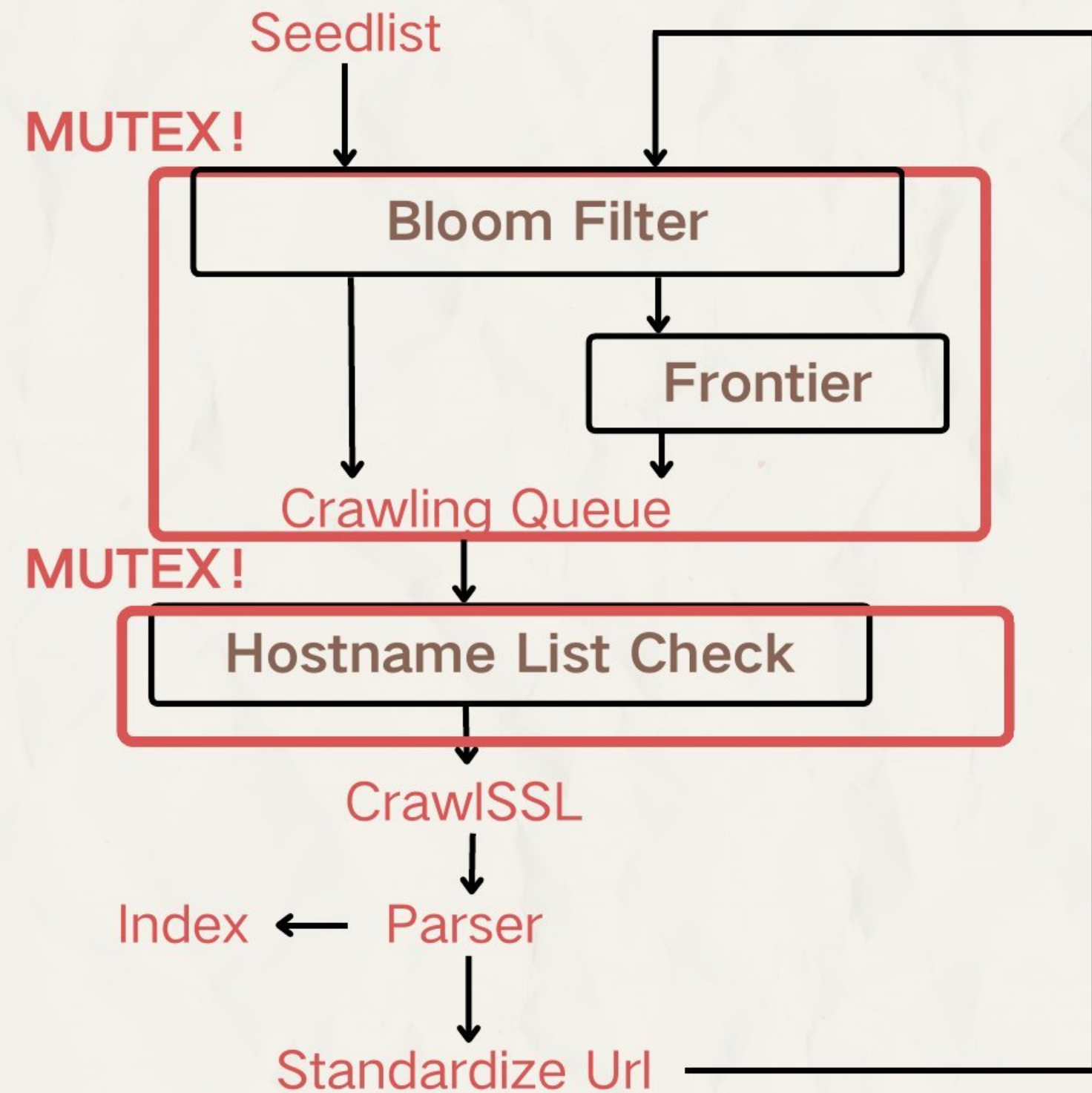


# Structure

single machine + single thread

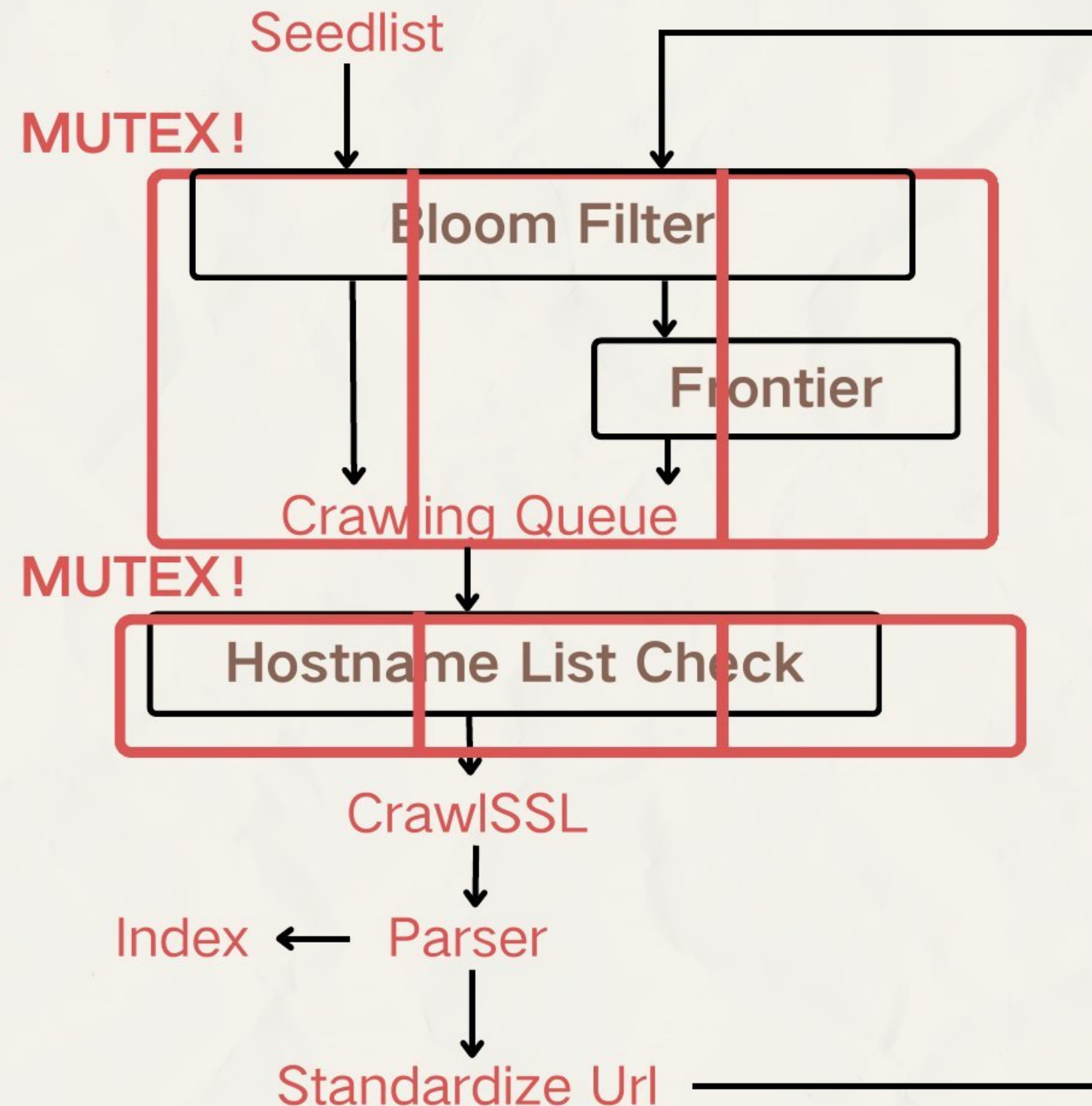


single machine + multi thread



# Structure

single machine + multi thread



## Problem:

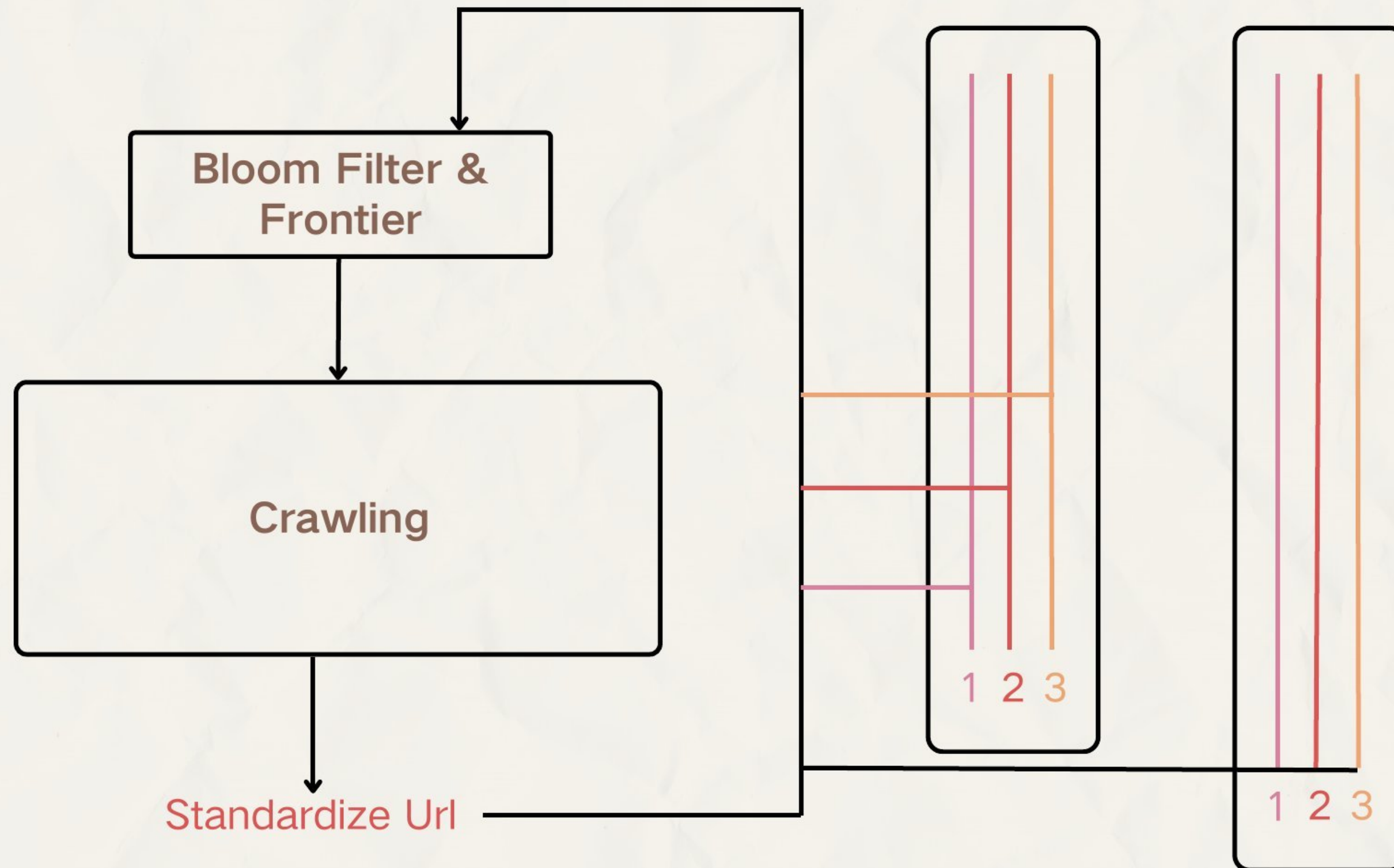
One big mutex -> The system not efficient when the worker thread++

## Solution:

- Split the Bloom Filter, Frontier, and Hostname List into partitions
- Each partition has its own Bloom Filter, Frontier, and Hostname List
- Workers are assigned to specific partitions
- More workers can run simultaneously

# Structure

single machine + multi thread



- **Hash** the url to put into the corresponding **buffer in sender**
- See if received something in receiver and **put back to frontier**

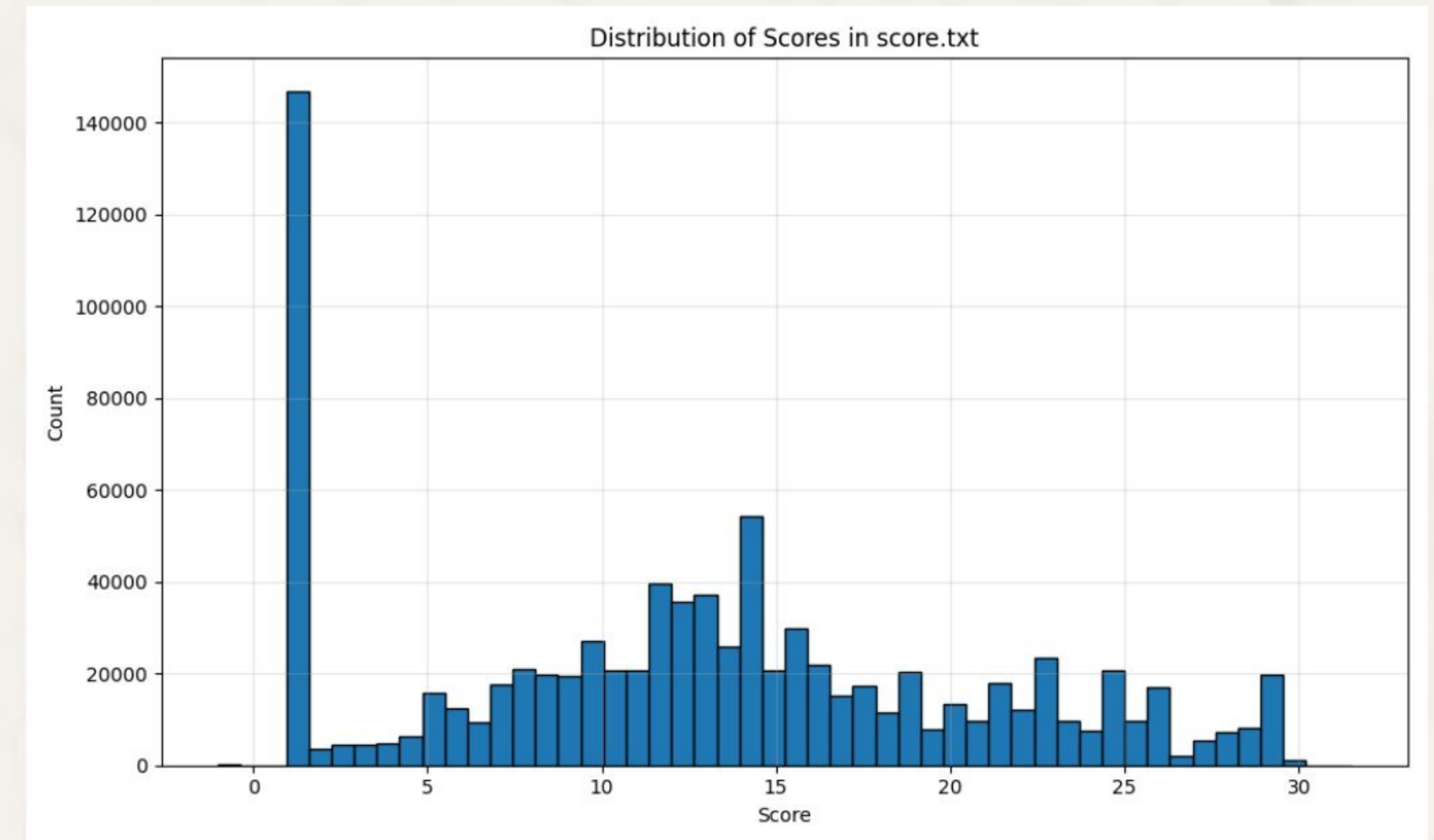
# URL Standardize & Score

## URL

- HTML entity decode
- http->https
- Search for redirect  
“u= ” / “redirect= ”
- Delete “.txt/.pdf...”
- .....

## Score

- Depth ★
- Https bonus
- Domain bonus
- Url length
- .....

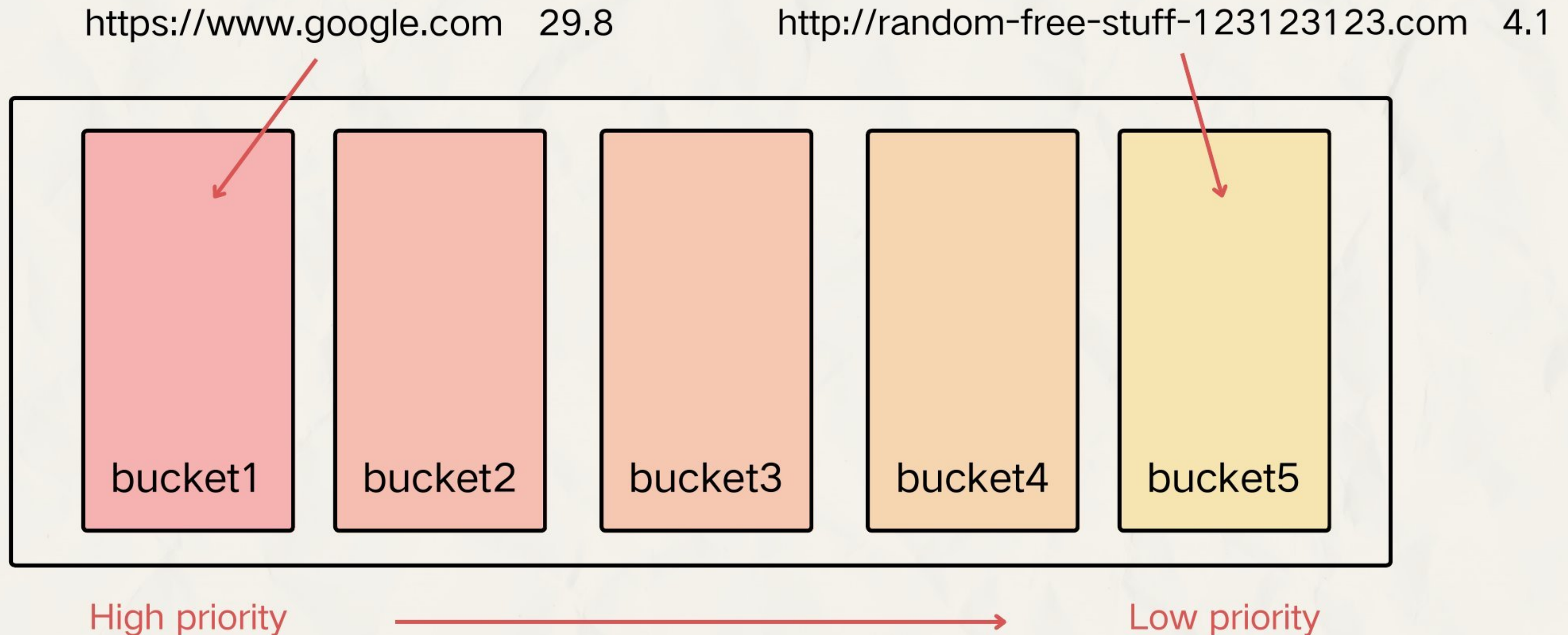


**Prune low-score URLs early to reduce storage and improve frontier quality!**

# Frontier

Multi buckets with different priorities

URLs are assigned to buckets based on their scores.



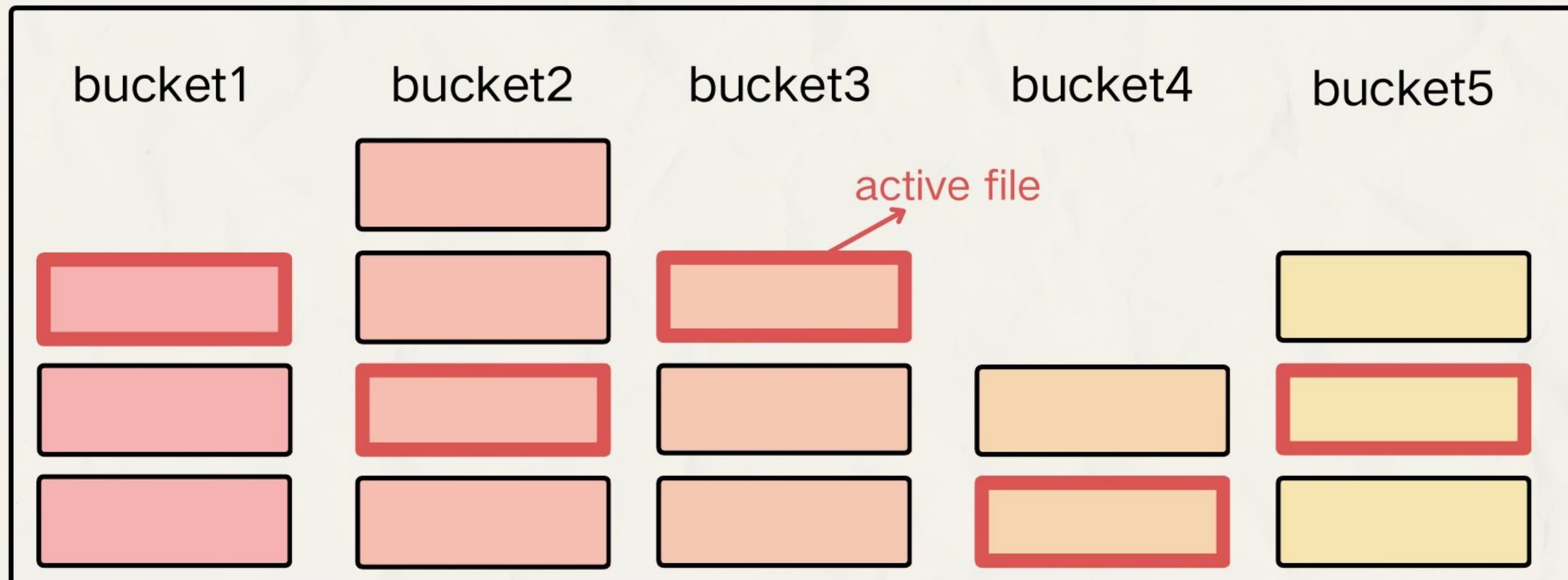
# Frontier

Multi files per bucket, one active file at a time

Only active file is in memory.

All insertion and extraction is applied to this active file.

**Reduce memory usage  
and I/O cost!**



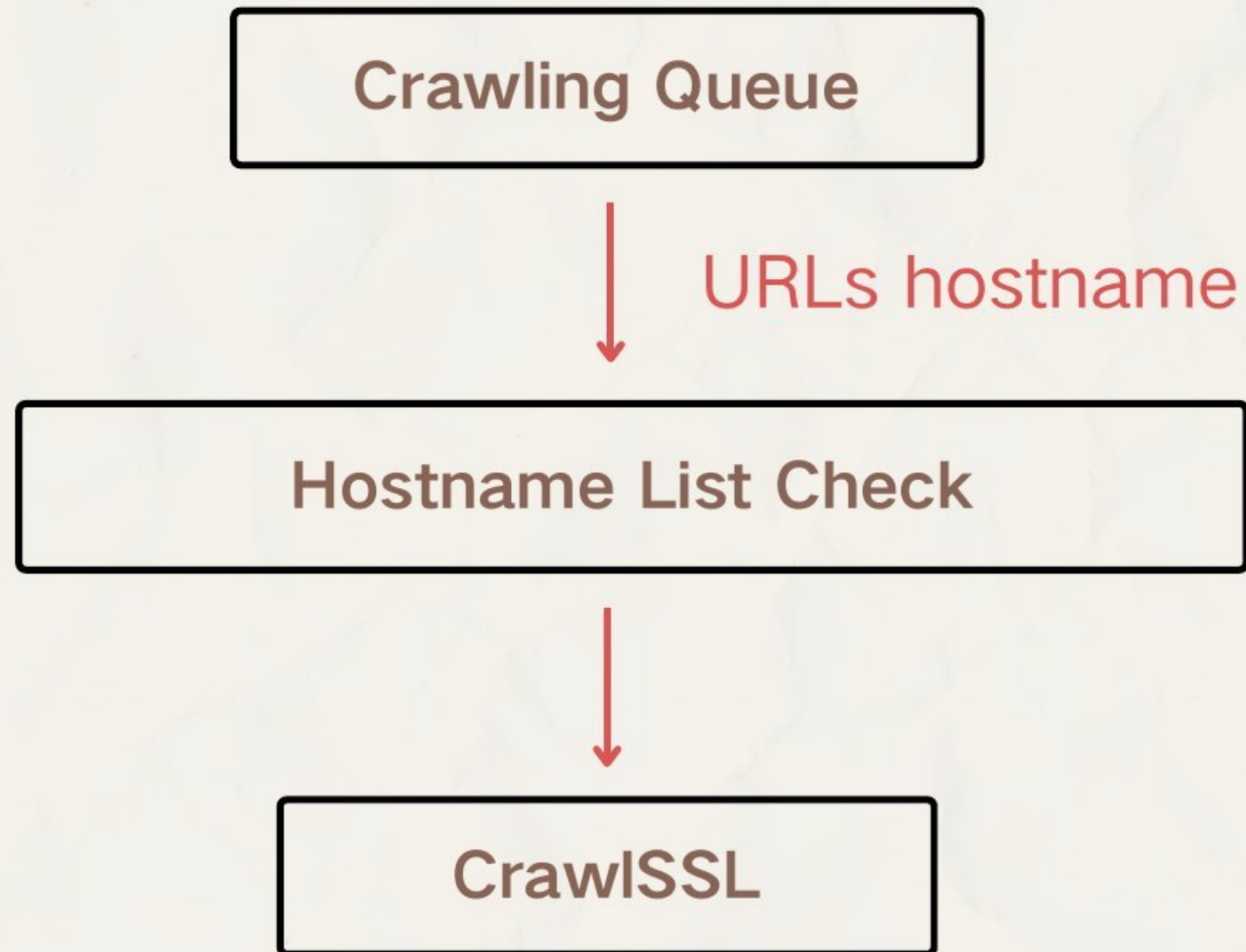
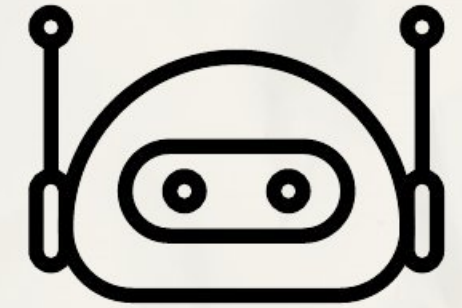
- Large file → split
- Small file → merge
- Too long time → rotates



Only flush when active  
file changes !!

# Hostname List

Store robots.txt and enforce crawl rules per hostname



Politeness Control  
robots.txt + crawl delay

Adaptive Backoff  
Increase delay for failed hosts (e.g. 503) 🙄

Memory Bounded  
Evict entries when full

# Parser

HTML page



Language Filtering



Content Extraction



Text Cleaning



- `<meta charset>`  
Only UTF-8, ISO-8859-1, Windows-1252
- `<html lang="...">`  
Only English pages
- Non-ASCII Filtering  
Dynamic threshold to limit uncommon char

- Original title
- Cleaned title + body
- Links and base URL

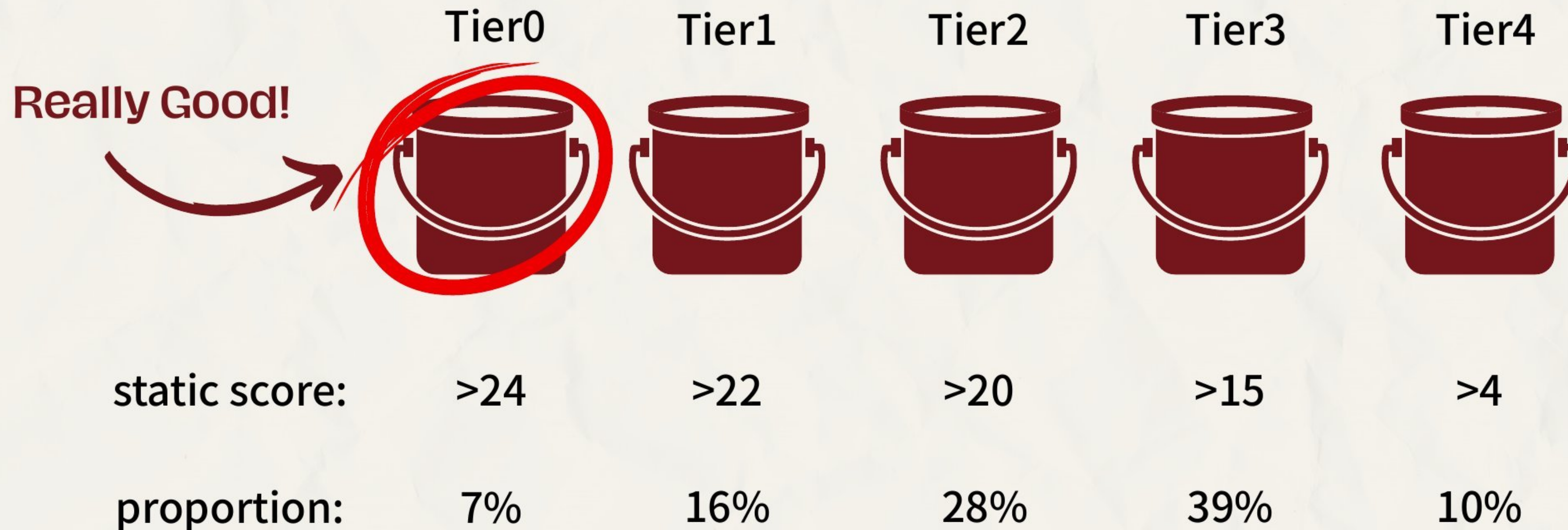
- Lowercases all alphabets
- Strips leading/trailing decorative chars  
@user → use, #tag → tag  
Keep special cases like C++
- Skips non-ASCII tokens
- Drops strange words  
>20 chars  
>4 special chars  
Pure numbers >5 digits  
Long (>10) alphanumeric ones

# Index Building Process Design

- Word decoration
  - For term umich
    - @umich if in title
    - %umich if in url
    - umich if in body
- URL token normalization
  - Split URL and remove noise
    - <https://umich.edu/student-life> → umich, edu, student, life
  - Split CamelCase
    - <https://ComputerScienceMajor> → computer, science, major
  - Word breaking based on dictionary
    - <https://machinelearningcourse> → machine, learning, course

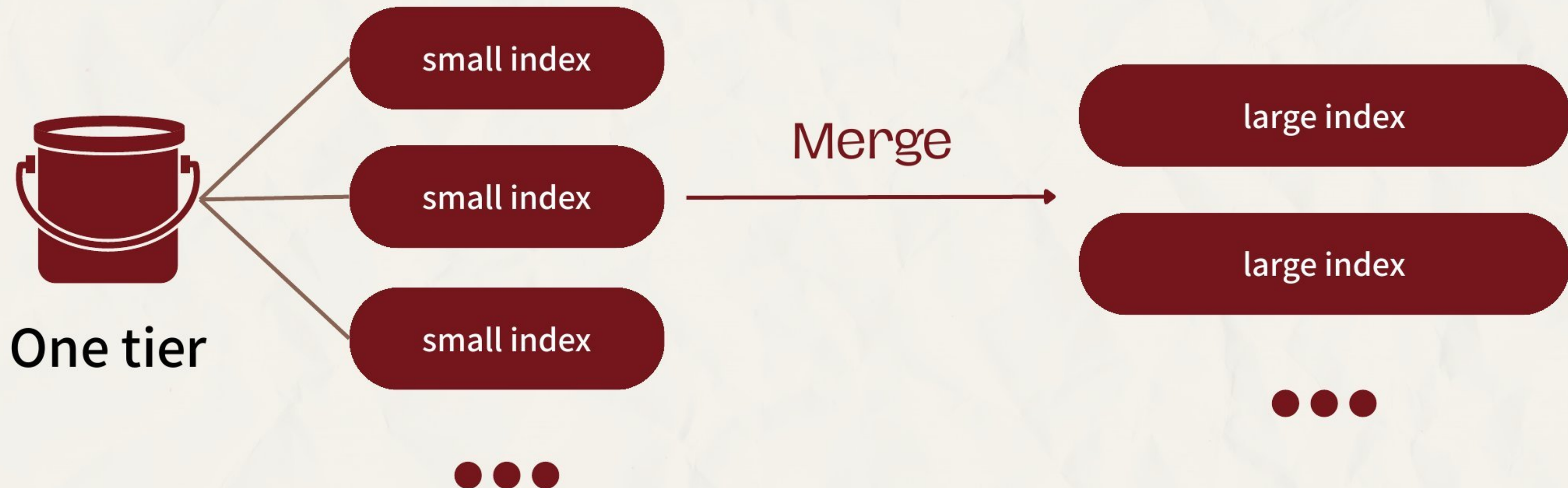
# Index Building Process Design

- Tier-based Index System
  - Pages are assigned to **different tiers** based on URL scores
  - Higher-tier index are prioritized during query time



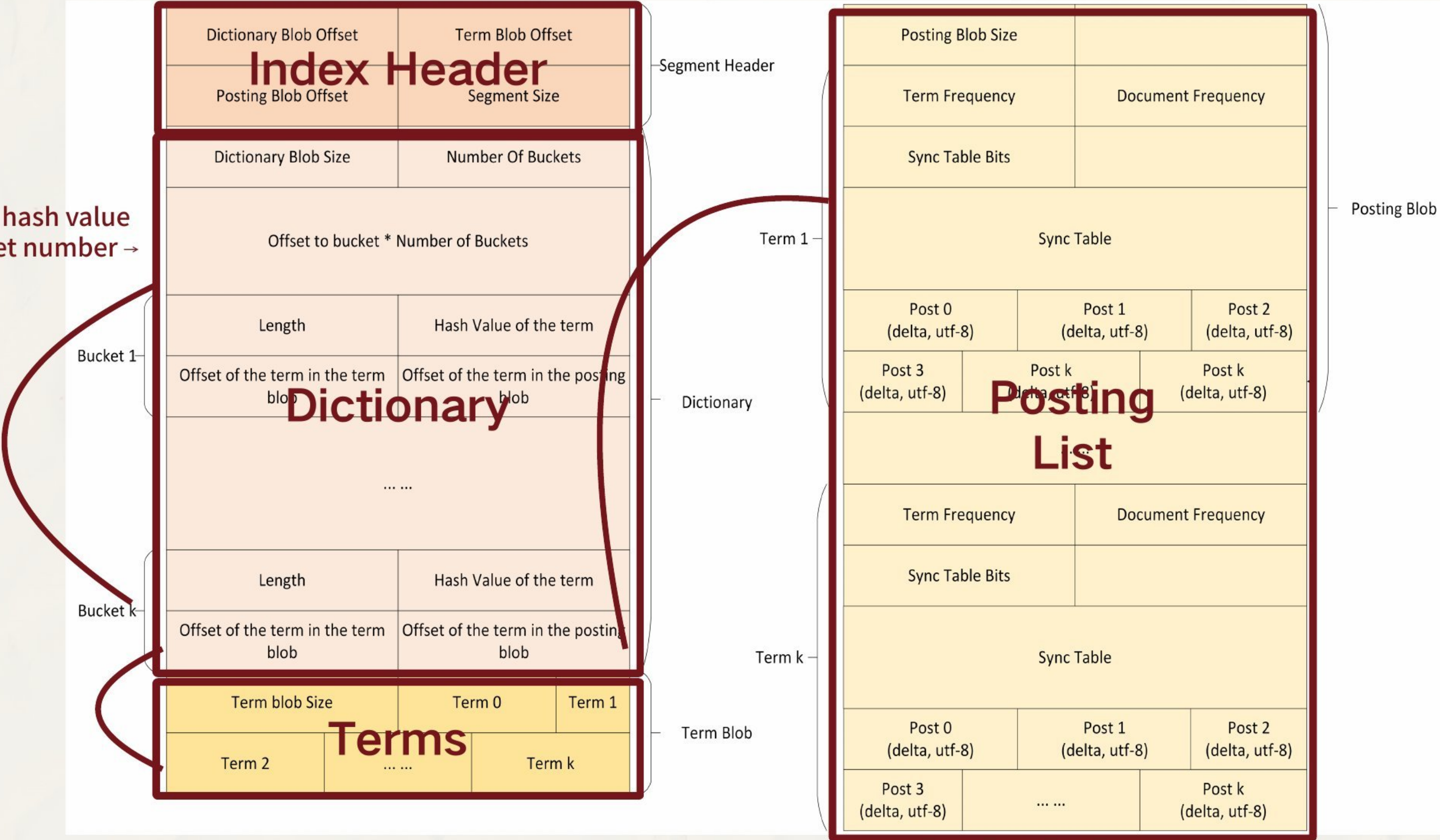
# Index Building Process Design

- Checkpoint & Flush
  - When we get 500 pages in hashtable, flush it to disk as a file
  - Can merge to bigger index ( **Tune index size & count for better search speed** )



# How to find a Term's posting list

Term → hash value  
→ bucket number →



# Index Design decisions

target = 1001 0110

slot =  
target >> (32 - N)=1

slot 1 →  
2nd entry in Sync Table

| SyncTable for SyncTable |     |        |    |                        |      |
|-------------------------|-----|--------|----|------------------------|------|
| 0                       |     | 2      |    | 12                     |      |
| slot 0                  |     | slot 1 |    | slot 2                 |      |
| SyncTable               |     |        |    |                        |      |
| entry #                 |     | absPos |    | offset in delta stream |      |
| 0                       |     | 10     |    | 0                      |      |
| 1                       |     | 90     |    | 2                      |      |
| 2                       |     | 150    |    | 6                      |      |
| ... ..                  |     |        |    |                        |      |
| Delta Stream            |     |        |    |                        |      |
| 10                      | +20 | +70    | +9 | ... ..                 | +202 |

- For size reduction
- delta encoding + sync table
  - utf-8 encoding

Fixed checkpoint every 64 term occurrence

# Index Design decisions

## Two-Level SyncTable Design:

### Prefix-based approach

- Index based on absolute position prefix
- Problem:
  - ignores term frequency (tf)
  - buckets can be very uneven
  - long delta scans → slow speed

**VS**

### Our SyncTable approach

- Insert checkpoint every 64 term occurrences
- Guarantees:
  - bounded scan ( $\leq 64$ ) in delta scan

**Search speed x 64 !!!**

# ISR - to find a matching page fast

- **ISRWord** for single term
- **ISRDocEnd** for document end and doc data
- **ISRAnd**, **ISRPhrase**, **ISROr**, **ISRContainer**

## DocEnd term design:



[docLen]  
[url]  
[title]  
[depth]  
...

**OldDesign:** [delta post] [docId]

**NewDesign:** [delta post] [docLen] [urlLen] [url] [titleLen] [title] [depth]



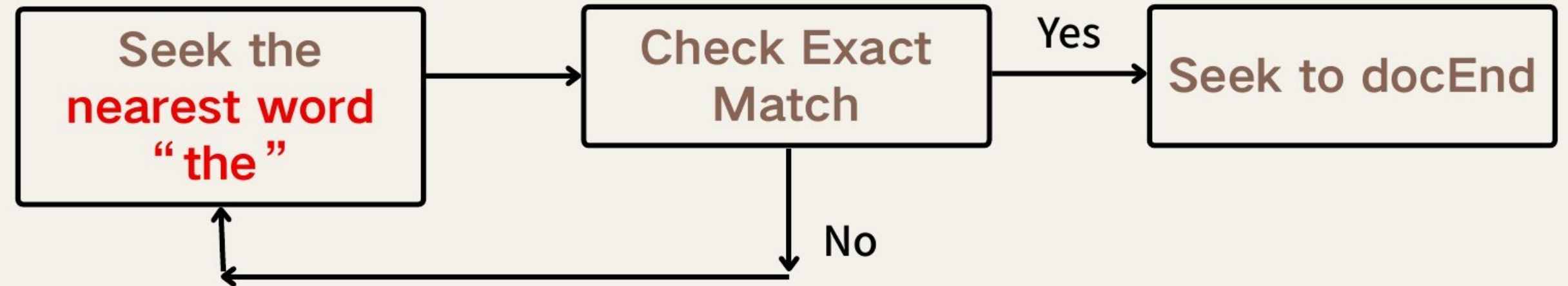
avoid extra disk IO

# ISR - Phrase search: "the quokka"

**Slow**



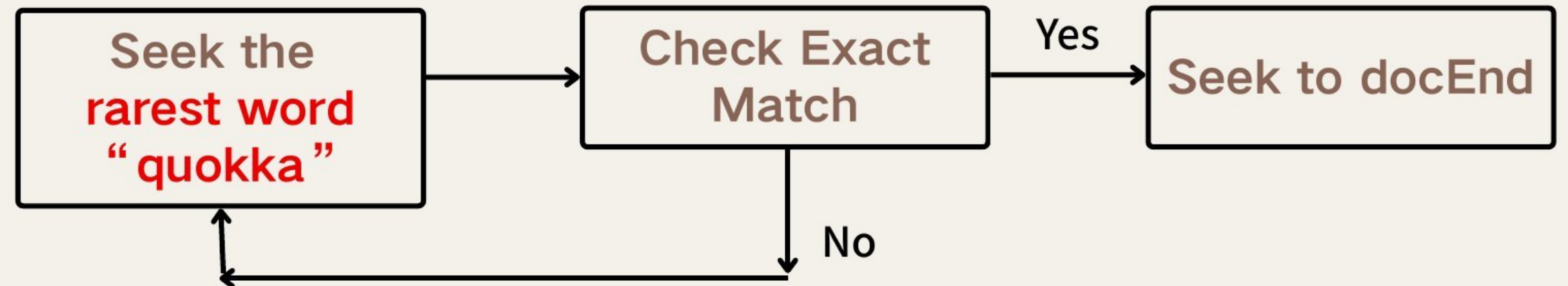
Too many candidates



**10x faster**



Few candidates



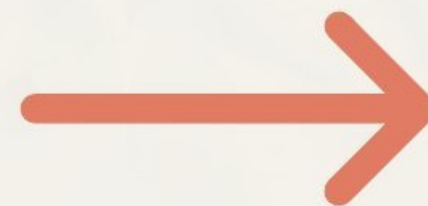
# Offline Deployment Result

- 5 Virtual Machine with 8 vCPU, 32G Memory(e2-standard-8)
- 30 million Index size ( Still Crawling lol)
- Query language: English

## System Capabilities

- Easy continue from crash
- One failure  $\neq$  system failure
- Real-time monitoring from anywhere

**Fantastic Monitoring  
Tools! !**



## Search bot

 CPU ALERT on ranker-e2-32g-1:  
0.50% < 10% for 30s 16:09

search 19:02 ✓✓

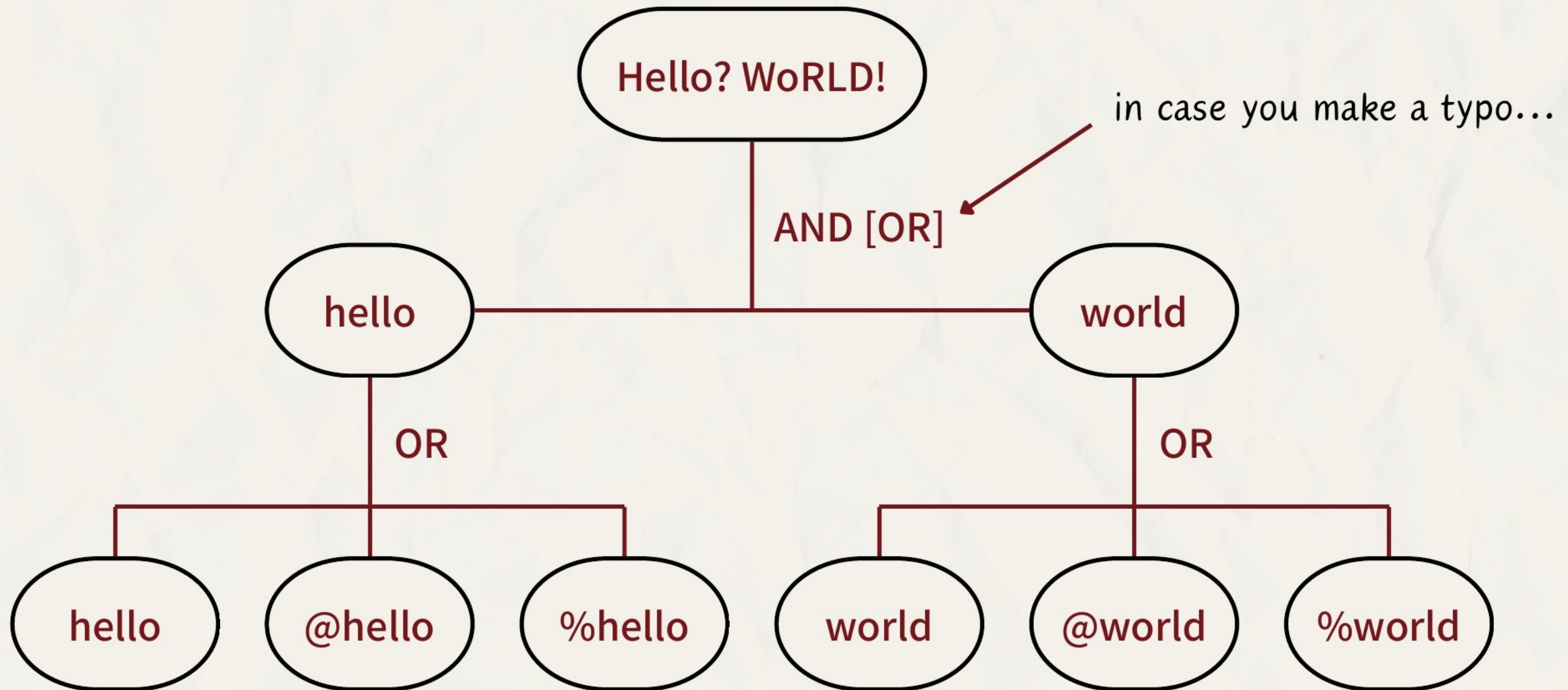
search result on ranker-e2-32g-3  
CPU: 3.76%  
last crawled url: [301489 707579](#)  
worker\_0 master.idx: size=495  
modified=2026-04-12  
22:33:12.171956149 +0000  
worker\_60 master.idx: size=864  
modified=2026-04-12  
23:02:26.902528037 +0000 19:02

search result on ranker-e2-32g-1  
CPU: 42.77%  
last crawled url: [205541 1205361](#)  
worker\_0 master.idx: size=748  
modified=2026-04-12  
22:55:08.790159336 +0000  
worker\_60 master.idx: size=680  
modified=2026-04-12

# **section 2**

**online query serving**

# From Query to ISR Tree



# Syntax List

|                |                              |
|----------------|------------------------------|
| "phrase words" | Exact consecutive phrase     |
| a b, a AND b   | Both must appear in same doc |
| a   b, a OR b  | Exact consecutive phrase     |
| NOT a, -a      | Exclude term from results    |
| ( expr )       | Exact consecutive phrase     |

# Ranking Heuristics - Static score



Yes:

- HTTPS
- root path
- .gov/.edu/umich.edu
- Wikipedia
- www. subdomain

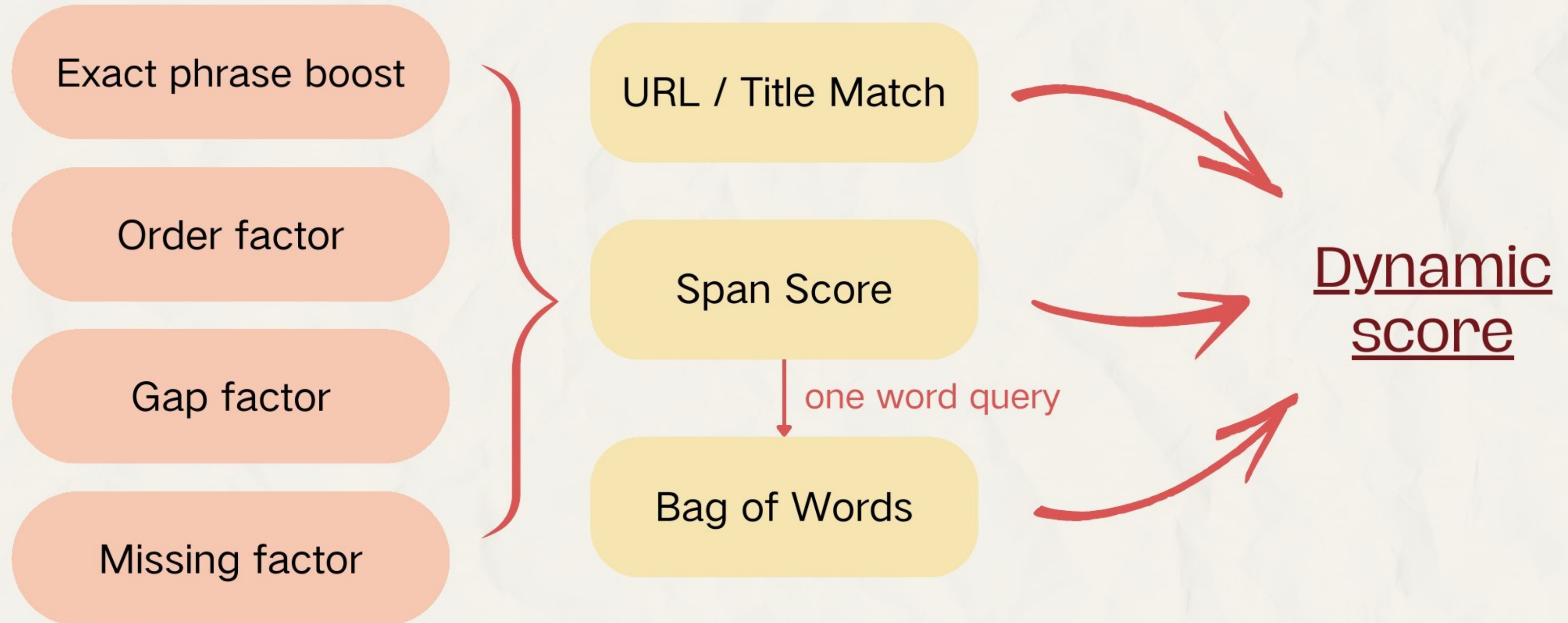


No:

- Large URL length,
- Far from seedlist
- Misc numbers in URL
- Long titles
  - Title > 200 chars are discarded

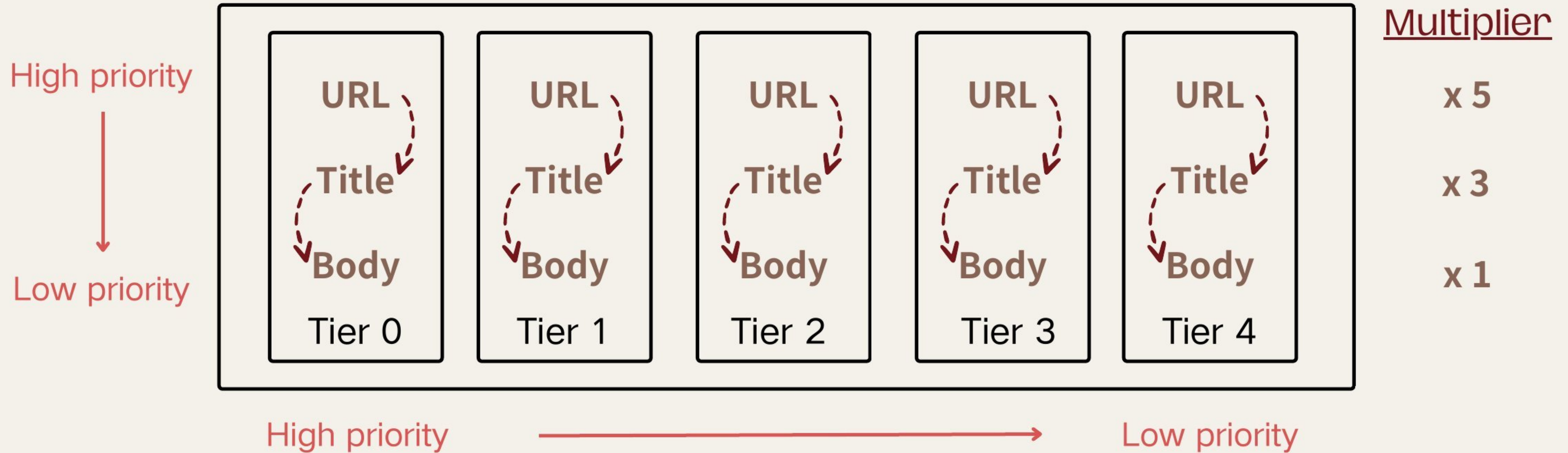
**Final score is a weighted sum of static  
and dynamic score**

# Ranking Heuristics - Dynamic score



# Ranking Strategy

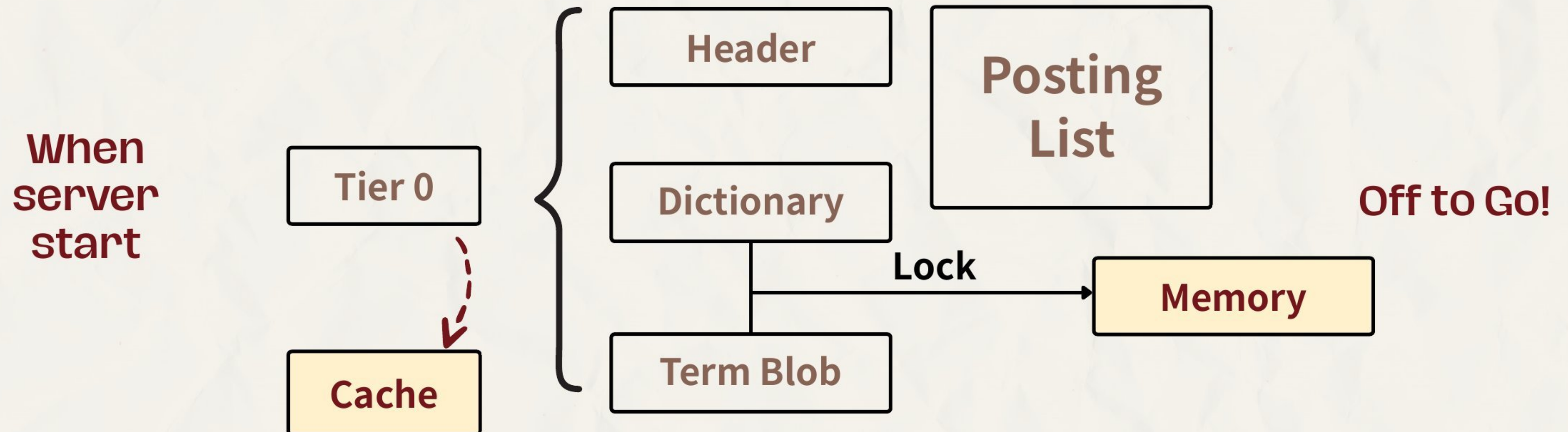
Tier-first, Three-pass Ranking



# Ranking Strategy

## Cache warming

- At the start of server, **mmap** the indexes into memory, set MAP\_POPULATE flag for index of **Tier 0** to trigger page fault for **cache** warming
- Lock the TermBlob and DictBlob of **Tier 0** index in **memory** (most frequently used!)



# Ranking Strategy

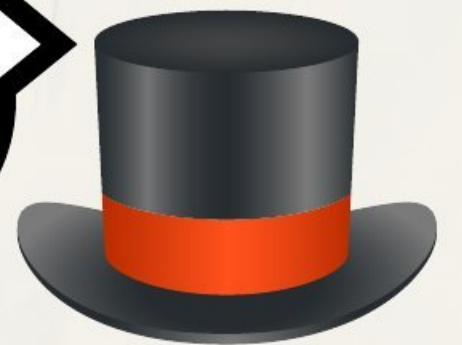
## Domain Capping

- Discount results in same domain ( $0.65 \times$  per additional result)
- Cap each domain for at most 3 results



Amazon?

amazon.com  
aws.amazon.com  
amazon.com.au/dp/B09XS7JWHH  
amazon.com/dp/B07Q9MJKBV  
amazon.co.uk/dp/B08HVZW4YK



# Ranking Strategy

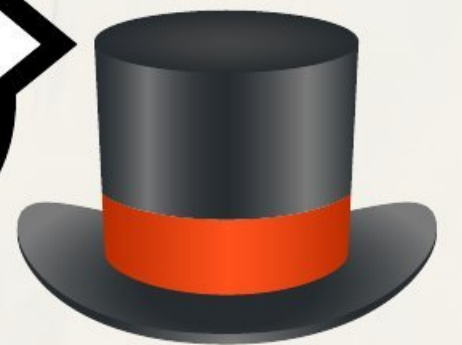
## Domain Capping

- Discount results in same domain ( $0.65 \times$  per additional result)
- Cap each domain for at most 3 results



How much do  
they pay you??

amazon.com  
aws.amazon.com  
amazon.com.au/dp/B09XS7JWHH  
amazon.com/dp/B07Q9MJKBV  
amazon.co.uk/dp/B08HVZW4YK



# Ranking Strategy

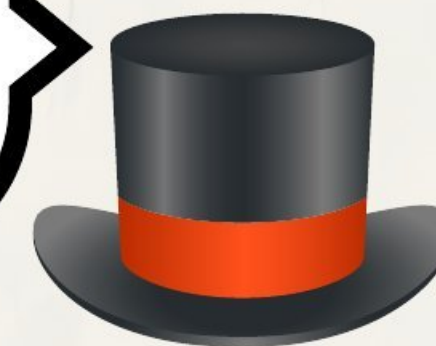
## Domain Capping

- Discount results in same domain ( $0.65 \times$  per additional result)
- Cap each domain for at most 3 results



Amazon?

amazon.com  
aws.amazon.com  
en.wikipedia.org/wiki/Amazon\_(company)  
facebook.com/Amazon  
primevideo.com



# Ranking Strategy

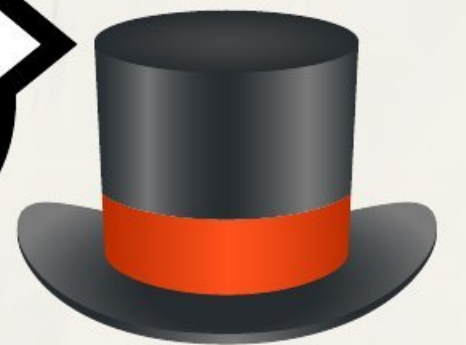
## Domain Capping

- Discount results in same domain ( $0.65 \times$  per additional result)
- Cap each domain for at most 3 results

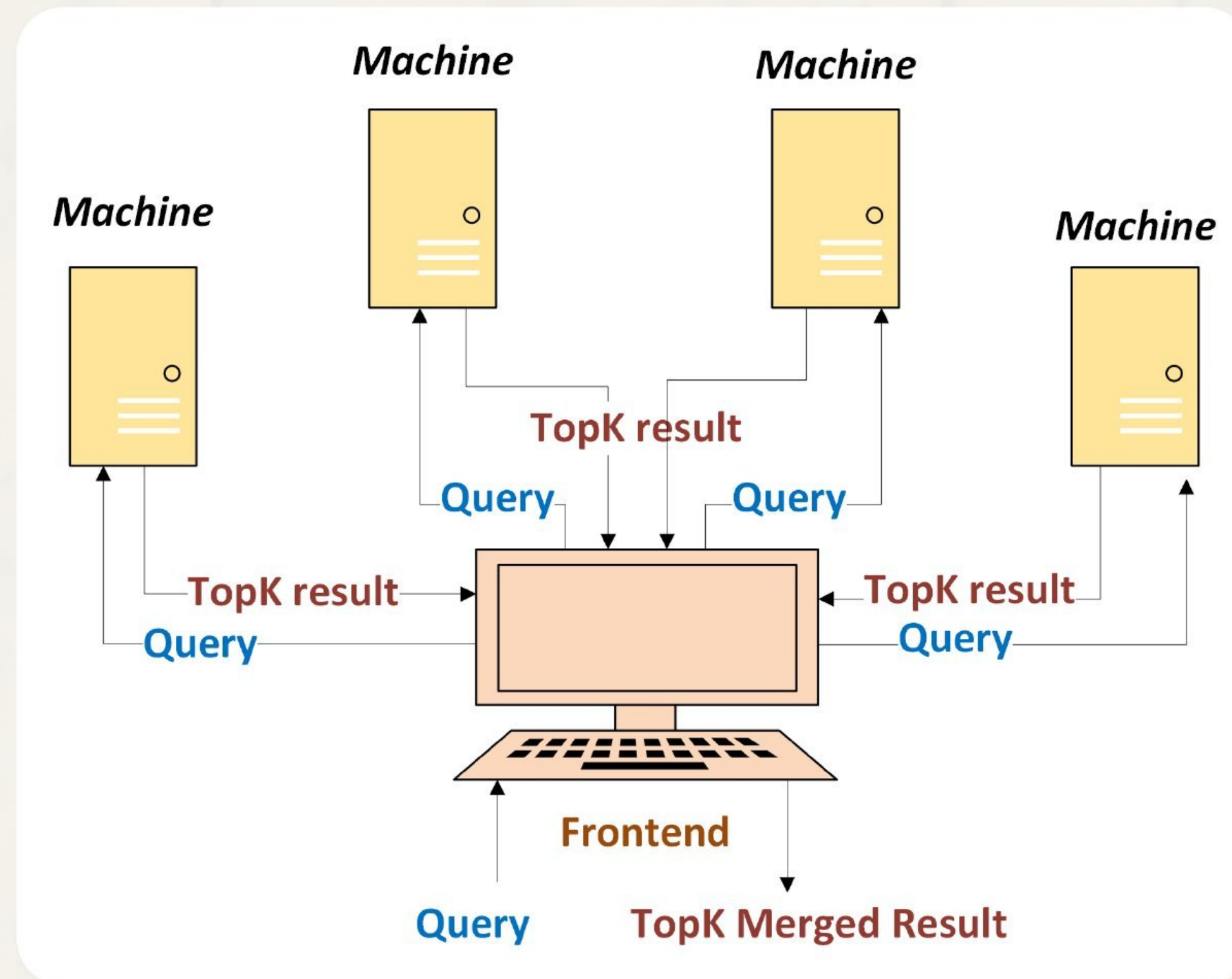


Great!

amazon.com  
aws.amazon.com  
en.wikipedia.org/wiki/Amazon\_(company)  
facebook.com/Amazon  
primevideo.com



# Ranking Strategy



Multi Machine Ranking

# The Ranking **Demo** That

# THANKS

## The Ranking Hat